

AIRFLOW PIPELINES, ENGINEERED

Airflow: DAG power, none of the daemon.

Why deploy a full service when you only need a robust library?
Redefining Micro-Orchestration. ⚡

+117k

downloads

<50 MB

RAM, total

SQLite

WAL state

THE SHIFT

Where it hurts today

Three things quietly cost teams time and trust every single week.

A metadata DB to babysit

DAGs drift from reality

CI for DAGs is painful



wpipe turns these into code you control.

What you gain



Code-first

Pipelines are Python — versionable, testable, reviewable.



Resume, don't restart

Every step checkpoints to SQLite WAL.



Featherweight

<50 MB RAM. No broker, no daemon.

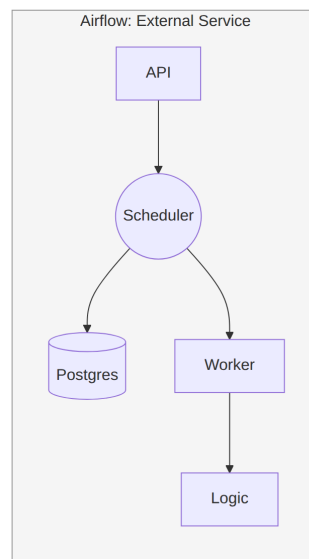
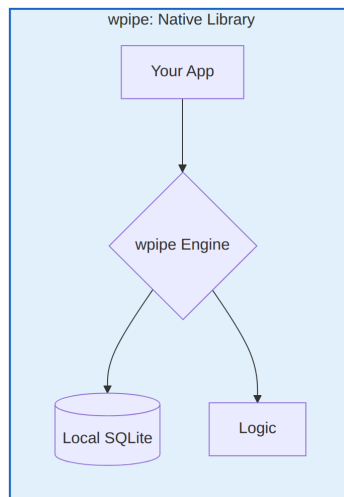
Crash at step 7? Resume from step 4.



Every step persists its state to SQLite WAL — a rerun cost you never pay again.

One pipeline, fully instrumented

State, retries, parallelism and audit — all inside the same run.



flow



wpipe vs. the status quo

Dimension	Apache Airflow	wpipe
Architecture	Client-Server (Centralized)	Native Library (Decentralized)
Footprint	GBs (Multiple containers)	MBs (Single process)
Persistence	External Postgres/Redis	SQLite WAL (Zero-Config)
Task Latency	Seconds (Scheduling overhead)	Milliseconds (Direct execution)
Maintenance	High (Cluster updates)	None (Dependency management)

Based on what the engineering community hits daily.

Answers four questions in one run ↘

Did it run, how far, what failed, and can it resume? Yes, yes, exactly, and automatically.

```
python my_pipeline.py
```

SEE IT IN ACTION

Six lines. That's the learning curve.

```
from wpipe import Pipeline, step

@step(name="job", retry_count=3, retry_delay=2)
def job(data):
    return {"ok": run(data["job"])}

pipe = Pipeline(pipeline_name="airflow_like")

CHECKPOINTS · RETRIES · TIMEOUT · PARALLEL · TRACKER
```



Code-first

Pipelines are Python — versionable, testable, reviewable.



Resume, don't restart

Every step checkpoints to SQLite WAL.



Featherweight

<50 MB RAM. No broker, no daemon.

Try wpipe in your next pipeline

Install the library, wrap your step, and watch checkpoints and docs appear by themselves.

```
pip install wpipe
```